Shai-Hulud: una minaccia nella supply chain JavaScript

Gruppo WannaCry

s358012 - Amedeo Marino
s361615 - Andrea Zanzarella
s356980 - Andrei Daniel Stefan
s361797 - Davide Giordano
s361817 - Leonardo Miani



Indice

1 Introduzione 2
2 Descrizione Shai-Hulud 2
3 Analisi Tecnica 3
4 Casi Reali 5
5 Conseguenze e Impatti 5
6 Mitigazioni e Comportamenti Corretti 6
7 Conclusioni e Discussione 6
Bibliografia 6

1 Introduzione

Un software moderno è molto spesso costruito riutilizzando codice scritto e testato in precedenza, sotto forma di librerie, in alcuni casi distribuite gratuitamente online su registri pubblici sotto forma di pacchetti. npm è una delle piattaforme di pacchetti per il linguaggio di programmazione JavaScript, nonché vettore di attacco del soggetto di questa relazione, il worm Shai-Hulud.

Un worm è un programma malevolo che ha la capacità di propagarsi autonomamente da un sistema all'altro. Shai-Hulud ricade sotto questa classificazione: tra il 14 e il 16 settembre 2025 (Pandya & Brown, 2025) ha infettato centinaia di pacchetti npm, diffondendo pubblicamente dati privati delle vittime. Il nome richiama i giganteschi vermi del deserto del romanzo **Dune** di Frank Herbert. Il malware pubblicava infatti i dati rubati in un nuovo progetto GitHub (una piattaforma online dedicata allo sviluppo software) pubblico con quel nome (Krebs, 2025).

2 Descrizione Shai-Hulud

Non è ancora confermato come il primo pacchetto sia stato infettato: è possibile collegare la diffusione del worm ad una campagna di phishing avvenuta pochi giorni prima (8 settembre 2025), dove le e-mail dell'attaccante, impersonando npm, richiedevano di aggiornare dati relativi all'autenticazione a due fattori (Krebs, 2025). In questo modo gli attaccanti avrebbero avuto accesso ai profili di diversi sviluppatori, e la possibilità di diffondere le prime copie di Shai-Hulud.

Questo tipo di attacco è diretto verso la supply-chain del software: non colpisce la vittima direttamente, ma un componente che utilizzerà di sua iniziativa, fidandosi in maniera quasi cieca. Quando uno sviluppatore installa un pacchetto infetto, il codice malevolo viene eseguito in fase di installazione, cercando sul computer credenziali per poterle rubare. Se Shai-Hulud trova anche un token npm, credenziali usate per pubblicare pacchetti a nome del proprietario, modifica i pacchetti della vittima e li ripubblica infetti, accelerando la propria diffusione.

Le vittime di questo attacco sono tutti gli sviluppatori che utilizzano JavaScript, anche fuori da contesti professionali: il linguaggio è utilizzato ampiamente anche in ambienti accademici e hobbistici. Le conseguenze ricadono anche su aziende che usano questi pacchetti e i sistemi di integrazione continua (CI), i server che compilano e pubblicano software in modo automatico. Shai-Hulud era progettato specificamente per Linux e macOS e non agiva su Windows (Kurmi, 2025; Krebs, 2025).

L'attacco non si basa su un singolo difetto, ma su alcune debolezze del sistema. La prima è la fiducia implicita: un pacchetto scaricato da npm viene eseguito senza particolari controlli, e l'installazione può avviare codice in automatico, cosa che il worm sfrutta per entrare in azione senza che lo sviluppatore se ne accorga (Pandya & Brown, 2025). A ciò si aggiunge il fatto che token e chiavi sono spesso facilmente raggiungibili sul computer dello sviluppatore e rubate con facilità. Infine mancano controlli sulla provenienza dei pacchetti: Weaver e il ricercatore Charlie Eriksen osservano che la pubblicazione può avvenire in modo automatico, senza il consenso esplicito di un umano (Krebs, 2025).

Tra gli asset in gioco, figurano:

- credenziali: token GitHub e npm, chiavi di accesso a cloud AWS, Google Cloud e Azure, e altre informazioni riservate lasciate nelle “variabili d’ambiente”, le impostazioni del sistema dove spesso si conservano chiavi (Kurmi, 2025);
- i progetti GitHub: il malware vi inserisce un automatismo che invia all’esterno i segreti del repository a ogni modifica (Kurmi, 2025);
- reputazione di aziende e progetti.

Anche se non confermato, il probabile attore dietro Shai-Hulud è TeamPCP, un gruppo cybercriminale spinto da motivazioni finanziarie. La stessa organizzazione è ritenuta responsabile delle successive ondate Shai-Hulud, pubblicando, nel maggio 2026 il codice sorgente del worm.

La sua diffusione è stata bloccata nel giro di 3 giorni, grazie da una serie di azioni: la pubblicazione online di elenchi dei pacchetti compromessi (Kurmi, 2025), la rimozione di questi dai registri npm (Krebs, 2025) e il blocco del traffico verso cui Shai-Hulud inviava parte dei dati rubati. Da parte delle vittime, è stato necessario eliminare i pacchetti infetti installati localmente e sostituire tutte le chiavi di accesso.

3 Analisi Tecnica

Le ricostruzioni indicano come punto di partenza una campagna di **phishing**: un’email mascherata da avviso di sicurezza di npm che spingeva gli sviluppatori ad “aggiornare” l’autenticazione a più fattori e inserire le credenziali su un sito controllato dall’attaccante. Con delle credenziali valide l’attaccante può pubblicare nuove versioni di pacchetti legittimi.

Il worm funziona in modo analogo a un virus: ogni vittima diventa a sua volta un infettore:

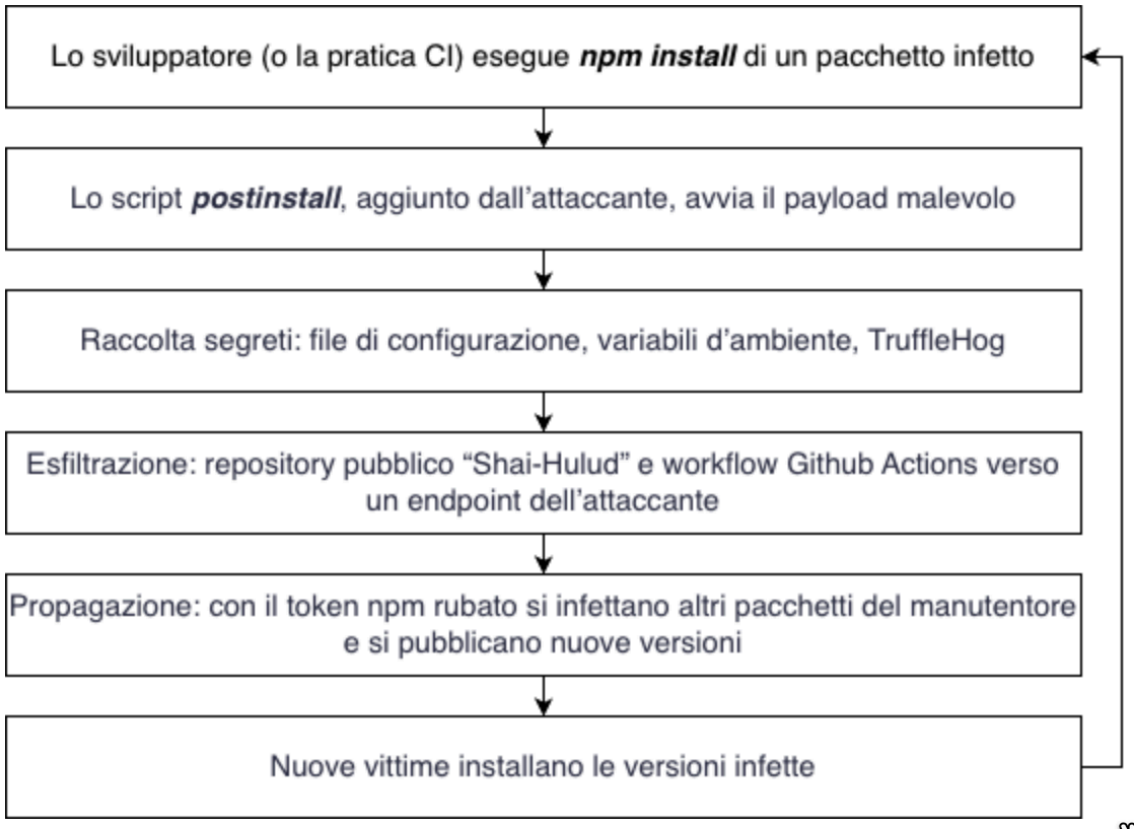


Figura 1: Propagazione di Shai-Hulud.

Nel dettaglio, si comporta così:

- **Esecuzione.** Il payload è un file javascript offuscato di circa 3,6 MB ([*bundle.js*](http://bundle.js)), lanciato dallo script **postinstall**. Il codice verifica di girare su Linux o macOS e salta i sistemi Windows.
- **Raccolta di segreti.** Il malware legge le variabili d'ambiente (compreso GITHUB_TOKEN), i token personali GitHub e i token npm; interroga i gestori di segreti di AWS e Google Cloud e i servizi di metadata delle istanze cloud; scarica ed esegue TruffleHog, un legittimo strumento open source di ricerca di segreti, sull'intero file-system. L'uso di uno strumento lecito, presentato come una «scansione di sicurezza», e la gestione silenziosa degli errori riducono le tracce lasciate.
- **Esfiltrazione.** Con il token GitHub trovato crea un repository pubblico chiamato Shai-Hulud nell'account della vittima, con un file JSON contenente informazioni di sistema, variabili d'ambiente e segreti. In più, aggiunge un branch con un workflow GitHub Actions (shai-hulud-workflow.yml) che, a ogni push, invia il contenuto di secrets a un endpoint webhook.site controllato dall'attaccante.
- **Esposizione di repository privati.** Il worm enumera i repository accessibili alla vittima, anche privati, e li rende visibili pubblicamente: Aikido lo descrive come un cambio di visibilità, TrendAI come la copia in repository pubblici marcati «Shai-Hulud Migration».
- **Propagazione.** Se trova un token npm valido (in ~/.npmrc o nell'ambiente), lo verifica, elenca i pacchetti di cui l'utente è manutentore (fino a 20), incrementa la versione,

aggiunge lo script postinstall e il file *bundle.js* e ripubblica. In questo modo l'infezione «salta» da un manutentore al successivo senza intervento umano.

4 Casi Reali

Il primo episodio documentato riguarda `@ctrl/tinycolor`, una libreria usata da un numero molto elevato di progetti e scaricata milioni di volte a settimana. Alcune sue versioni sono state modificate con codice malevolo che si avviava automaticamente durante l'installazione (Pandya et al., 2025). Per chi installava la dipendenza non c'era nulla di anomalo da notare: il comando era quello abituale. È proprio questo che rende il caso istruttivo, perché la fiducia verso una dipendenza molto diffusa è bastata a portare il codice malevolo dentro i sistemi. Il danno non era solo nella macchina infetta: il codice cercava credenziali e le inviava a server esterni o le pubblicava in repository GitHub pubblici (Cohesity REDlab, n.d.). Una credenziale esposta può quindi portare a nuovi accessi, ben oltre il singolo computer.

Nella stessa ondata sono stati colpiti anche pacchetti pubblicati da CrowdStrike, una delle aziende di cybersecurity più note (Pandya et al., 2025). Il fatto che un'azienda che si occupa di sicurezza sia finita nella catena suggerisce che l'attacco non dipendesse dalla competenza di chi pubblicava: ciò che contava era l'accesso all'account. Non abbiamo prove che l'attaccante abbia scelto CrowdStrike per il suo prestigio, ma il caso mostra che la reputazione di un editore non basta a proteggere chi scarica i suoi pacchetti.

Il numero più citato è quello dei pacchetti compromessi. La CISA parla di oltre 500 pacchetti nel suo alert del 23 settembre 2025 (CISA, 2025). GitHub ha dichiarato di averne rimossi più di 500 e di aver bloccato il caricamento di nuove versioni che contenevano gli stessi indicatori di compromissione [1]. La risposta mostra la sequenza tipica di un incidente di questa scala: prima si eliminano i pacchetti infetti, poi si impedisce che tornino con le stesse tracce. Il numero, però, dice poco da solo: ciò che conta è che bastano poche ore perché un singolo punto d'ingresso raggiunga centinaia di progetti.

Nel novembre 2025 il worm è tornato in una forma più aggressiva, ribattezzata Shai-Hulud 2.0. Il ricercatori di Datadog hanno contato circa 800 pacchetti npm compromessi e hanno osservato un cambiamento importante: il worm non si limitava più a eseguire codice durante l'installazione, ma installava sulla macchina della vittima un *runner* self-hosted di GitHub Actions, cioè un agente che permette di eseguire comandi da remoto (DATADOG Security Labs, 2025). In pratica, la macchina di uno sviluppatore, o un server di build, diventava una porta di accesso permanente per l'attaccante. Il bersaglio non è più solo chi installa un pacchetto, ma anche l'infrastruttura che produce il software distribuito agli utenti.

Nel 2026 il meccanismo è stato riutilizzato da altri attori. Il 1° giugno 2026 un pacchetto di 32 librerie pubblicate sotto il namespace `@redhat-cloud-services` è stato compromesso con una variante chiamata Miasma, che Red Hat identifica come una versione di Mini Shai-Hulud (Red Hat, 2026).

5 Conseguenze e Impatti

TODO

6 Mitigazioni e Comportamenti Corretti

Le principali azioni compiute da Shai-Hulud possono essere riassunte in due punti principali:

1. **Esfiltrazione di credenziali**: il worm aggiunge al file `package.json` del pacchetto `npm` infettato due nuovi script per inizializzare ed eseguire il runtime JavaScript «Bun», potendo eseguire il payload bypassando i controlli di `node.js`. La credenziali vengono cercate sia nel filesystem locale che in cloud, e poi pubblicate in una repository pubblica su GitHub.
2. **Replicazione automatica**: se il worm individua un token `npm` valido salvato in locale (ad esempio nel file `.npmrc`), interroga l'API di `npm` per identificare fino a 100 pacchetti mantenuti dall'utente. Dopodiché scarica tali pacchetti, inietta i propri script di `preinstall` nel `package.json`, incrementa la versione patch e ripubblica automaticamente i pacchetti infetti tramite `npm publish`.

Una prima contromisura da adottare per prevenire l'attivazione del worm è aggiungere il flag `--ignore-scripts` in fase di installazione delle dipendenze (con `npm install`): in questo modo, lo script `preinstall` non verrà eseguito. Sempre in ambito dipendenze, è opportuno cancellare la cache `npm` e modificare il `package.json` «pinnando» le dipendenze a una versione nota rilasciata in data precedente all'attacco.

Per prevenire l'ulteriore propagazione del worm ed evitare la compromissione dei propri account, ciascun utente dovrebbe abilitare servizi di autenticazione a più fattori, idealmente tramite *chiavi di sicurezza hardware* (es., YubiKey). In questo modo, quando il malware tenta di completare la chiamata API per il `npm publish`, la piattaforma rifiuta la richiesta per mancanza della firma crittografica forte o dell'OTP hardware valido, bloccando sul nascere la catena di infezione della supply chain.

7 Conclusioni e Discussione

Il caso Shai-Hulud dimostra che nella supply chain del software la fiducia è un punto critico: bastano le credenziali di pochi sviluppatori, tramite phishing, per infettare milioni di sistemi. L'esecuzione automatica del codice all'installazione senza lettura ne conferma è una falla nel design di npm gravissima, esposta solo da attacchi di queste dimensioni. Il contenimento è stato possibile grazie alla reazione rapida di npm, dei ricercatori e delle aziende coinvolte, ma, come testimoniano i successivi attacchi lanciati verso npm, la piattaforma continua a rimanere fragile e un ottimo mezzo di propagazione.

Bibliografia

[1] Jenna Barron, «GitHub details upcoming changes to improve security in wake of Shai-Hulud worm in npm ecosystem». Consultato: 15 ottobre 2026. [Online]. Disponibile su: https://sdtimes.com/security/github-details-upcoming-changes-to-improve-security-in-wake-of-shai-hulud-worm-in-npm-ecosystem/